

GPU Programming(2026) Assignment 3: Parallel Single-Source Shortest Path (SSSP) via Static and Adaptive Δ -Stepping on GPUs

Department of Computer Science & Engineering, IIT Madras

Assignment Submission Guidelines

1. Fill your code in main.cu file.
 2. Compile it with : `nvcc main.cu -o main`
 3. Run the `run_testcases.sh` script to test your code on all the public testcases given in input and output folders.
 4. Submit it as `ROLL_NO_ALL_CAPS.cu` on moodle.
 5. **Note:** For using using Bellman-Ford Parallel implementation, you will not get full marks. Though it is good to use it for generating your own testcases and check your code.
 6. **Note:** You can use Stl (if not comfortable with Thrust library. Though a small tutorial is given in Appendix) for this assignment.
 7. **Recursive Note:** Please read the note at the end too. That one is more important.
-

1 Problem Overview & Motivation

The Single-Source Shortest Path (SSSP) problem is a fundamental graph problem. Given a directed graph $G = (V, E)$ with non-negative integer edge weights $w : E \rightarrow \mathbb{Z}_{\geq 0}$ and a designated source vertex $s \in V$, the objective is to compute the minimum path weight from s to every vertex $v \in V$.

Standard algorithms represent two design extremes:

- **Dijkstra's Algorithm:** Processes vertices in strict total distance order using a priority queue. While work-efficient ($O(|E| + |V| \log |V|)$), its step-by-step dependency chain offers virtually no parallelism, making it poorly suited for GPU execution.
- **Bellman-Ford Algorithm:** Relaxes all $|E|$ edges concurrently over $|V| - 1$ iterations. Although it exposes massive data parallelism, it suffers from severe work inflation and excessive atomic memory contention due to redundant relaxations.

Δ -Stepping bridges this gap by discretizing distances into ordered buckets of width $\Delta > 0$:

Bucket k covers the distance interval $[k \cdot \Delta, (k + 1) \cdot \Delta]$

This structure allows vertices within the same distance bucket to be relaxed concurrently on GPU threads, striking a balance between memory efficiency and hardware saturation.

Algorithm 1: Parallel Bellman-Ford SSSP Algorithm

Input: Directed Graph $G = (V, E)$, edge weights $w : E \rightarrow \mathbb{Z}_{\geq 0}$, Source vertex $s \in V$
Output: Array $\text{dist}[0 \dots |V| - 1]$ containing shortest path distances

```

// Initialization Phase
1  $\text{dist}[s] \leftarrow 0$ 
2 for each  $v \in V \setminus \{s\}$  do
3    $\text{dist}[v] \leftarrow \infty$ 
4  $\text{updated} \leftarrow \text{true}$ 

// Iterative Parallel Edge Relaxation
5 while  $\text{updated}$  do
6    $\text{updated} \leftarrow \text{false}$ 
7   for each vertex  $u \in V$  in parallel do
8     if  $\text{dist}[u] \neq \infty$  then
9       for each outgoing edge  $(u, v) \in E$  do
10         $\text{new\_dist} \leftarrow \text{dist}[u] + w(u, v)$ 
11        // Concurrent update via atomic minimum
12        if  $\text{new\_dist} < \text{dist}[v]$  then
13          if  $\text{AtomicMin}(\text{dist}[v], \text{new\_dist}) > \text{new\_dist}$  then
14             $\text{updated} \leftarrow \text{true}$ 
15 return dist

```

2 Architectural Foundations

2.1 Standard Δ -Stepping Mechanics

Edges are classified based on the parameter Δ :

- **Light Edges** ($w(e) \leq \Delta$): Relaxing a light edge from a node in bucket k can produce a target node that still falls inside bucket k . Thus, light edge relaxations run iteratively in an inner loop until bucket k is completely drained.
- **Heavy Edges** ($w(e) > \Delta$): Relaxing a heavy edge guarantees the target node lands strictly in a future bucket $> k$. Heavy edges originating from all nodes settled during bucket k 's light phase are relaxed once per bucket step.

3 Memory Optimization: Modular Circular Bucket Ring

In a naive implementation of Δ -stepping, each logical bucket $0, 1, 2, \dots, k$ is assigned a dedicated buffer or array in memory. However, for graphs with a large diameter or high edge weights, the maximum logical bucket index $k_{\max} = \lceil D_{\max}/\Delta \rceil$ can grow into the thousands or millions. Allocating physical GPU memory for every logical bucket leads to catastrophic VRAM fragmentation and out-of-memory errors.

3.1 The Bounded Horizon Property

To eliminate unbounded allocation, we exploit a structural invariant of shortest-path edge relaxation:

Theorem 1 (Bounded Horizon Property). *Let D_{curr} be the minimum tentative distance at the start of a bucket phase, and let Δ be the active bucket width. If a vertex u is processed within the active bucket such that $tent[u] \leq D_{curr} + \Delta$, then relaxing any light edge $e = (u, v)$ with weight $w(e) \leq \Delta$ yields an updated tentative distance bounded by:*

$$tent[v] = tent[u] + w(e) \leq D_{curr} + 2\Delta$$

Furthermore, any vertex v promoted into the active Near worklist for the current bucket phase satisfies the inclusive boundary condition:

$$tent[v] \leq D_{curr} + \Delta$$

Proof. Let $u \in \text{Near}$ be an active vertex extracted during the current bucket step. By definition of bucket extraction via the cutoff threshold $\text{cutoff} = D_{curr} + \Delta$, we have $tent[u] \leq D_{curr} + \Delta$.

For any light edge $e = (u, v)$, the edge weight satisfies $w(e) \leq \Delta$. Upon relaxation, the candidate distance assigned to v is:

$$tent_{\text{new}}[v] = tent[u] + w(e)$$

Substituting the upper bounds for $tent[u]$ and $w(e)$:

$$tent_{\text{new}}[v] \leq (D_{curr} + \Delta) + \Delta = D_{curr} + 2\Delta$$

If $tent_{\text{new}}[v] \leq D_{curr} + \Delta$, vertex v falls directly into the current right-inclusive bucket $[D_{curr}, D_{curr} + \Delta]$ and is processed in the immediate light iteration. If $D_{curr} + \Delta < tent_{\text{new}}[v] \leq D_{curr} + 2\Delta$, vertex v is deferred to the Far queue and is guaranteed to be settled within at most $\lceil (D_{curr} + 2\Delta)/\Delta \rceil$ subsequent bucket steps, preserving algorithmic correctness and finite termination. $\square$

3.2 Physical Slot Mapping via Modulo Arithmetic

Because active vertices never span further than W_{max} beyond the active bucket, the maximum number of concurrently required physical bucket buffers is strictly bounded by M :

$$M = \left\lceil \frac{W_{\text{max}}}{\Delta} \right\rceil + 1$$

Instead of allocating k_{max} global arrays, the system allocates a fixed array of exactly M physical bucket slots in GPU global memory. Any logical bucket index k_{target} maps to its corresponding physical slot $S \in \{0, 1, \dots, M-1\}$ using modulo arithmetic:

$$S = k_{\text{target}} \pmod{M}$$

3.3 Slot Lifecycle and Memory Reuse Invariants

The modular circular bucket ring operates under three core rules:

1. **Target Slot Calculation:** When relaxing an edge (u, v) that produces an updated distance $tent[v]$, compute the target logical bucket index:

$$k_{\text{target}} = \left\lfloor \frac{tent[v]}{\Delta} \right\rfloor$$

The target vertex v is pushed directly into physical slot array $S = k_{\text{target}} \pmod{M}$.

2. **Drain and Recycling:** When logical bucket k completes its light and heavy relaxation phases, all vertices assigned to physical slot $S_{\text{curr}} = k \pmod{M}$ have been fully settled.
3. **Reclamation Guarantee:** By the Bounded Horizon property, no active vertex in the system can map to slot S_{curr} again until logical bucket $k + M$ is reached. Therefore, physical slot S_{curr} can be atomically reset (queue count set to 0) and immediately reused for logical bucket $k + M$.

This modular mapping guarantees that the total GPU memory footprint for queue storage remains $O(M + |V|)$, completely independent of graph diameter, path lengths, or maximum execution steps.

4 Theoretical Analysis: Impact of the Δ Parameter

The performance and execution pattern of the Delta-Stepping algorithm are strongly governed by the bucket width parameter Δ . By tuning Δ , the algorithm spans a spectrum between two classic SSSP algorithms: **Sequential Dijkstra’s Algorithm** and **Parallel Bellman-Ford**.

1. Large Δ Special Case ($\Delta \geq w_{\max}$)

When Δ is chosen such that it is greater than or equal to the maximum edge weight in the graph ($\Delta \geq \max_{e \in E} w(e)$):

1. **Absence of Heavy Edges:** By definition, an edge (u, v) is heavy if $w(u, v) > \Delta$. Since $w(u, v) \leq w_{\max} \leq \Delta$ for all edges, **every edge in the graph is classified as a Light Edge**. The heavy edge relaxation phase processes zero edges.
2. **Unconstrained Bucket Expansion:** When a vertex $u \in \text{Near}$ relaxes an edge (u, v) , the new distance satisfies:

$$tent[v] \leq tent[u] + w(u, v) \leq \text{current_min_d} + \Delta$$

Consequently, vertex v is immediately placed into the Near queue rather than deferred to the Far queue.

3. **Degeneration to Parallel Bellman-Ford / Level-Synchronous BFS:** The algorithm processes large frontiers in parallel waves. While this maximizes GPU hardware occupancy, it sacrifices strict distance ordering. A vertex may be relaxed multiple times in the same bucket phase, leading to redundant edge relaxations.

2. Small Δ Special Case ($\Delta \rightarrow 0$ or $\Delta = 1$)

When $\Delta = 1$ (assuming unit or small integer weights):

1. **Minimal Bucket Width:** Nodes are partitioned into narrow distance buckets of width 1.
2. **Degeneration to Dijkstra’s Algorithm:** Vertices are processed in strict non-decreasing order of their shortest path distances.
3. **Work Efficiency vs. Parallelism Trade-off:** No vertex is relaxed more than once (zero duplicate work), but the available parallelism per phase is extremely low, causing low GPU thread utilization.

3. Summary of Asymptotic Bounds

The choice of Δ controls the trade-off between **work efficiency** (total relaxations) and **parallelism** (step complexity):

Metric	Small Δ ($\Delta = 1$)	Optimal Δ	Large Δ ($\Delta \geq w_{\max}$)
Algorithm Behavior	Dijkstra-like	Balanced Delta-Stepping	Parallel Bellman-Ford
Parallelism (GPU Utilization)	Low	High	Maximal
Redundant Relaxations	None (0%)	Minimal	High
Step Complexity	$\mathcal{O}(V)$	$\mathcal{O}\left(\frac{ V }{\Delta} + \text{Diam}(G)\right)$	$\mathcal{O}(V)$

Key Takeaway for Students: The goal of choosing Δ (or using an adaptive scheme with parameter K) is to find the sweet spot where the Near queue contains enough vertices to saturate CUDA thread blocks without doing excess non-shortest-path work.

5 Memory Optimization: Near/Far Two-Queue Abstraction

Rather than allocating an unbounded ring array of explicit buckets in GPU VRAM, this assignment uses a streamlined **Near/Far Two-Queue Design**:

1. **Near_Queue (Active Worklist):** Holds vertex IDs currently eligible for processing in the active distance window $[D_{\text{curr}}, D_{\text{curr}} + \Delta]$.
2. **Far_Queue (Overflow Pool):** A single global array storing all active nodes whose tentative distances exceed $D_{\text{curr}} + \Delta$.
3. **Queue State Array (queue_state[v]):** Tracks vertex membership (`STATE_NONE = 0`, `STATE_NEAR = 1`, `STATE_FAR = 2`) to prevent duplicate queue insertions during parallel relaxation.

When **Near_Queue** becomes empty, a stream compaction/filter kernel scans **Far_Queue**, promoting nodes with $\text{tent}[v] < D_{\text{curr}} + \Delta$ into **Near_Queue**.

6 Adaptive Δ -Stepping Mechanism

Fixed choices of Δ present a trade-off: a value too small underutilizes the GPU hardware, while a value too large causes atomic write contention and redundant relaxations.

Adaptive Δ -stepping calculates Δ dynamically at every round transition to maintain a target GPU hardware saturation of $K = 65,536$ active vertices:

1. **Parallel GPU Sort:** At the start of a transition phase, `Far_Queue` is sorted by tentative distance $tent[v]$ using `thrust::sort_by_key`.
2. **Minimum Distance Identification:** Set $D_{\min} = tent[\text{sorted_far}[0]]$.
3. **Handling Duplicate Keys via Binary Search:** Inspect the distance value $V = tent[\text{sorted_far}[K-1]]$ at the boundary capacity index. To ensure that identical distance values are not split across rounds, use `thrust::lower_bound` to locate the first occurrence index I_{safe} of V .
4. **Dynamic Cutoff & Δ Calculation:**

$$\Delta_{\text{adaptive}} = tent[\text{sorted_far}[I_{\text{safe}} - 1]] - D_{\min}$$

(Fallback: If even index 0 contains $> K$ duplicate values, use `thrust::upper_bound` to process all identical minimal entries together with $\Delta = 0$).

5. **Promotion:** Transfer the top I_{safe} sorted elements directly into `Near_Queue`.

—

7 Input and Output Specifications

7.1 Input Format (`input.txt`)

The input is provided as a whitespace-separated sequence of integers via standard input ('stdin'):

```
<delta> <K>
<N> <E> <S>
<src_0> <src_1> ... <src_S-1>
<offsets[0]> <offsets[1]> ... <offsets[N]>
<neighs[0]> <neighs[1]> ... <neighs[E-1]>
<weights[0]> <weights[1]> ... <weights[E-1]>
```

- `<delta> <K>` (int, int):
 - **Static Mode:** $\text{delta} \geq 0$, $K = -1$ (placeholder).
 - **Adaptive Mode:** $\text{delta} = -1$, $K > 0$ (target batch capacity, e.g., 65536).
- `<N> <E> <S>` (int, int, int): Total vertices ($0 \leq v < N$), total directed edges, and the number of source queries S .
- `<src_0> ... <src_S-1>` (S integers): List of source vertex indices to evaluate sequentially.
- `<offsets>`, `<neighs>`, `<weights>` ($N+1$, E , and E integers): Graph structure in CSR format.

7.2 Output Format (`stdout`)

Strict Output Protocol: Output sent to standard output (`stdout`) must contain **strictly numeric text (integers)**. Do not print text labels, string prefixes, debugging output, system headers, or empty lines between source outputs.

The program must process all S source queries sequentially in the exact order specified in the input array (`src0, src1, ..., srcS-1`). For each source query `srci` ($0 \leq i < S$), the output must adhere to the following sequence:

1. **Adaptive Δ Log Entries (Only when delta == -1):** At every transition step for source src_i where `Near_Queue` is empty and a dynamic batch is promoted from `Far_Queue`, **print the calculated integer Δ on its own line before** launching the relaxation kernels for that batch.
2. **Final Shortest Path Distances (Always):** Print exactly N lines corresponding to the output for source src_i . Line j ($0 \leq j < N$) must contain the shortest path distance from src_i to vertex j . If vertex j is unreachable from src_i , print 2147483647 (`INT_MAX`).

Across all S sources, an execution in Adaptive mode will produce a total of $\sum_{i=0}^{S-1} (\text{Num_Transitions}_i + N)$ output lines, whereas Static mode will produce exactly $S \times N$ output lines.

8 Mathematical Safety Proof: Bounded Distance Representation

Theorem 2 (Distance Boundedness). *For any directed graph $G = (V, E)$ satisfying $|V| \leq 2,000,000$ and non-negative edge weights $w(e) \leq 1,000$, the shortest path distance $d(s, v)$ from any source $s \in V$ to any reachable vertex $v \in V$ strictly fits within a 32-bit signed integer without overflow, leaving $\text{INT_MAX} = 2,147,483,647$ uniquely reserved as an unreachable sentinel value.*

Proof. Because all edge weights are non-negative ($w(e) \geq 0$), any optimal (shortest) path $P = (v_0, v_1, \dots, v_k)$ from source $s = v_0$ to target $v = v_k$ is a simple path containing no cycles. Therefore, path P contains at most $k \leq |V| - 1 = 1,999,999$ directed edges.

The upper bound for shortest path length $D_{\max}$ occurs in a worst-case linear chain topology where $k = N - 1$ and every edge assumes the maximum possible weight $W_{\max} = 1,000$:

$$D_{\max} = \sum_{i=1}^k w(v_{i-1}, v_i) \leq (N - 1) \cdot W_{\max}$$

Substituting the maximum system constraint parameters yields:

$$D_{\max} \leq (2,000,000 - 1) \cdot 1,000 = 1,999,999,000$$

Comparing this theoretical bound against the 32-bit signed integer limit:

$$D_{\max} = 1,999,999,000 < 2,147,483,647 = \text{INT_MAX}$$

Thus, $d(s, v) < \text{INT_MAX}$ holds for all reachable vertices $v \in V$. Integer overflow is mathematically impossible under these constraints, and `INT_MAX` can be safely utilized to represent unreachable vertices ($d(s, v) = \infty$) without numerical collision. $\square$

8.1 Justification of Technical Constraints

Vertices ($N \leq 2 \cdot 10^6$): Provides a massive active frontier (5 million vertices) that exposes warp divergence and grid saturation dynamics. This scale forces dynamic worklist algorithms to maintain high warp execution efficiency while ensuring queue arrays (‘tent’, worklists, state masks) remain manageable within VRAM.

Edges ($E \leq 300 \cdot 10^6$): Pushes global VRAM utilization to ≈ 2.6 GB for graph structures alone (1.2 GB for neighbor indices and 1.2 GB for edge weights). This massive edge capacity transforms SSSP into a realistic memory-bandwidth-bound workload, thoroughly stress-testing GPU L2 cache eviction behaviors, memory coalescing, and DRAM throughput across 300 million edge relaxations.

Edge Weights ($w(e) \leq 1000$): Tightly bounds maximum edge weights to guarantee that the absolute worst-case shortest-path distance ($D_{\max} = (5,000,000 - 1) \times 400 = 1,999,999,600$) remains strictly less than `INT_MAX` = 2,147,483,647. This enables native 32-bit GPU atomic primitives (`atomicMin`) and 4-byte memory alignments without risking integer overflow or incurring the 64-bit atomic performance penalties.

Source Queries ($S \leq 20$): Tests algorithm stability and convergence efficiency across diverse graph entry points (e.g., highly connected hubs versus sparse peripheral chains) while maintaining autograder runtime bounds.

Sentinel (2147483647): Standard 32-bit signed integer maximum (`INT_MAX`). With valid maximum path lengths bounded at $\approx 1.999 \times 10^9$, this value serves as a mathematically collision-free sentinel representing unreachable nodes ($d(s, v) = \infty$).

VRAM Allocation Policy: With total VRAM requirements exceeding 2.6 GB, calling `cudaMalloc` and `cudaFree` inside the multi-source loop would trigger catastrophic host-device synchronization bottlenecks and device memory fragmentation. Mandating a single allocation at program startup forces clean memory reuse via fast initialization kernels or `cudaMemset`.

9 Technical Constraints

See Table 1 for the constraints.

Table 1: System Constraints and Technical Parameters

Parameter / Resource	Constraint Value
Number of Vertices (N)	$1 \leq N \leq 2,000,000$
Number of Edges (E)	$0 \leq E \leq 300,000,000$
Edge Weight Range (w)	$0 \leq w(e) \leq 1,000$ (32-bit non-negative integers)
Adaptive Batch Capacity (K)	$1 \leq K \leq 524,288$ (active vertex threshold for <code>delta == -1</code>)
Unreachable Node Sentinel	2147483647 (<code>INT_MAX</code>)
Allowed CUDA Libraries	Thrust Primitive Library (<code>thrust::sort_by_key</code> , etc.)
Expected Peak VRAM	≈ 2.6 GB (allocated once at startup)

10 Sample Execution Scenarios

The following test scenarios demonstrate the exact input and standard output formats for both Static and Adaptive modes on a 5-vertex, 6-edge directed graph.

10.1 Scenario A: Static Mode ($\delta \geq 0$, $K = -1$, Multi-Source $S = 2$)

Sample Input (input_static.txt)

```
2 -1
6 7 2
1 3
0 2 4 5 7 7 7
1 2 2 3 4 4 5
2 5 1 4 3 1 2
```

Expected Standard Output (stdout)

```
1 // src_0
2147483647
0
1
4
4
6
3 // src_1
2147483647
2147483647
2147483647
0
1
2
```

10.2 Scenario B: Adaptive Mode ($\delta = -1$, Target Capacity $K = 2$, Multi-Source $S = 2$)

Sample Input (input_adaptive.txt)

```
-1 2
6 7 2
1 3
0 2 4 5 7 7 7
1 2 2 3 4 4 5
2 5 1 4 3 1 2
```

Expected Standard Output (stdout)

```
1 // src_0
3 // delta logging
1
2147483647
0
1
4
4
6
```

```
3                // src_1
1                // delta logging
2147483647
2147483647
2147483647
0
1
2
```

Note:

- Half of the test cases will be for static delta and adaptive delta each.
- If you use Bellman-ford to solve the problem, you will automatically fail half of the testcases, hence 6 marks at most.
- For static delta part, you can also use Modular Circular Bucket Ring technique also(just for fun). We have set the constraints such that you may not need more than 11 or 12GB of Global Memory. So, you have enough flexibility to run it on colab and Aqua anyway as 32GB GPUs.
- For adaptive delta part, be careful. Since delta-stepping is a very asynchronous or self-correcting code, you might always get the correct short distances at the end, but the sequence of deltas might change on every run. So, check your own implementation whether it produces the same output everytime or not.

Thanks for Reading till here.

Please remember to delete the // Generated by ChatGPT comments before submitting.

11 Deterministic K -Adaptive Δ -Stepping SSSP Implementation Guide

This document details four critical implementation edge cases ("catches") in parallel adaptive Δ -stepping on GPU architectures. Strict adherence to these rules is required to ensure deterministic trace log generation and pass automated evaluation.

11.1 1. Re-anchoring Minimum Distance ($D_{\min}$)

Core Rule At the beginning of Phase 3 (Queue Refill and Bucket Decision), $D_{\min}$ (`current_min_d`) must always be set directly to the smallest key currently remaining in the **Far** queue:

$$D_{\min} = \text{far_keys}[0] \quad (1)$$

Do **not** carry over the previous iteration's cutoff distance (`last_cutoff`) as $D_{\min}$.

Concrete Example Suppose Iteration 1 finished with `last_cutoff` = 10. After edge relaxations, the remaining queue in Iteration 2 is:

$$\begin{aligned} \text{far_queue} &= [2, 5, 8] \\ \text{far_keys} &= [18, 22, 25] \quad \text{with } K = 2 \end{aligned}$$

- **Correct Implementation** ($D_{\min} = \text{far_keys}[0]$):

$$\begin{aligned} D_{\min} &= 18 \\ D_{\text{cutoff}} &= \text{far_keys}[1] = 22 \\ \Delta &= 22 - 18 = 4 \quad (\text{Correct distance spread across } K \text{ vertices}) \end{aligned}$$

- **Flawed Implementation** ($D_{\min} = \text{last_cutoff}$):

$$\begin{aligned} D_{\min} &= 10 \\ D_{\text{cutoff}} &= 22 \\ \Delta &= 22 - 10 = 12 \quad (\text{Artificially inflated across empty gap } [10, 18]) \end{aligned}$$

Impact Carrying over `last_cutoff` across distance gaps inflates the reported bucket width Δ . While the final shortest path distances ($\text{tent}[v]$) remain mathematically correct, delta inflation breaks adaptive sampling logic and causes trace log divergence against evaluator ground truth.

11.2 2. Delta Calculation and Trace Output ($\Delta = 0$ Plateaus)

Core Rule Given the sorted **Far** queue size $|\text{Far}|$ and sampling parameter K :

1. Sample target index: $i_{\text{target}} = \min(K - 1, |\text{Far}| - 1)$.
2. Read bucket cutoff distance: $D_{\text{cutoff}} = \text{far_keys}[i_{\text{target}}]$.
3. Compute raw delta: $\Delta_{\text{raw}} = D_{\text{cutoff}} - D_{\min}$.
4. Write Δ_{raw} directly to the output trace file (even if $\Delta_{\text{raw}} = 0$).
5. If $\Delta_{\text{raw}} == 0$, set the internal kernel parameter `active_delta` = 1 for Phase 1 light-edge relaxation ($w \leq \text{active_delta}$) to prevent infinite loops.

Concrete Example Consider a graph state with parameter $K = 2$ where multiple vertices share identical distances (a distance plateau):

$$\begin{aligned}\text{far_queue} &= [1, 2, 3, 4] \\ \text{far_keys} &= [12, 12, 12, 15]\end{aligned}$$

1. $D_{\min} = \text{far_keys}[0] = 12$.
2. $i_{\text{target}} = \min(2 - 1, 4 - 1) = 1 \implies D_{\text{cutoff}} = \text{far_keys}[1] = 12$.
3. $\Delta_{\text{raw}} = 12 - 12 = 0$.
4. **Trace Output:** Write "0" to the log file.
5. **Internal Execution:** Set $\text{active_delta} = 1$ internally so that zero-weight or unit light edges can still be processed in Phase 1 without getting stuck.

11.3 3. Candidate Extraction via Right-Inclusive upper_bound

Core Rule To populate the **Near** queue from the **Far** queue, perform a right-inclusive search up to D_{cutoff} using `thrust::upper_bound`:

$$\text{candidate_count} = \text{thrust::upper_bound}(\text{far_keys}, \text{far_keys} + |\text{Far}|, D_{\text{cutoff}}) - \text{far_keys} \quad (2)$$

All vertices v satisfying $\text{tent}[v] \leq D_{\text{cutoff}}$ must be extracted into the **Near** queue in a single pass. Do **not** truncate extraction strictly at K elements.

Concrete Example Suppose $K = 2$, $D_{\min} = 4$, and four vertices are tied at distance 8:

$$\begin{aligned}\text{far_queue} &= [10, 11, 12, 13, 14, 15] \\ \text{far_keys} &= [4, 8, 8, 8, 8, 15]\end{aligned}$$

Sampling at $K = 2$ gives $D_{\text{cutoff}} = \text{far_keys}[1] = 8$.

- **Right-Inclusive Extraction (upper_bound on key 8):** Finds the first key strictly greater than 8 (at index 5).

$$\text{Extracted to Near} = \{10, 11, 12, 13, 14\} \quad (\mathbf{5} \text{ vertices extracted, } \mathbf{5} > \mathbf{K})$$

Trace Log: Single $\Delta = 4$ entry logged. The plateau is cleared in 1 pass.

- **Flawed Strict Truncation at $K = 2$:** Extracts only $\{10, 11\}$, leaving $\{12, 13, 14\}$ behind in **Far**. **Trace Log:** Requires a second pass to clear the remaining tied nodes, logging an extra $\Delta = 0$ entry (Trace : $[4, 0]$), causing autograder failure.

11.4 4. Ghost Vertex Detection and Queue Pruning

Core Rule At the start of Phase 3, before selecting $D_{\min}$ or calculating Δ , prune all stale "ghost" entries from the **Far** queue:

$$\text{Prune all entries where } \text{far_keys}[j] \leq \text{last_cutoff} \quad (3)$$

Concrete Example Suppose in Iteration 1, $\text{last_cutoff} = 10$, extracting nodes $\{1, 2\}$ into **Near**. Node 3 remains in **Far** with initial distance 15.

1. **Phase 1 Relaxation (Iteration 2):** Node 1 relaxes edge $1 \rightarrow 3$ with weight $w = 2$.

$$\text{tent}[3] = 4 + 2 = 6$$

Since $6 \leq \text{last_cutoff}$ (10), Node 3 is pulled directly into **Near** and processed.

2. **Phase 2 Relaxation (Iteration 2):** Node 1 relaxes heavy edge $1 \rightarrow 4$ ($w = 12$). Distance $\text{tent}[4] = 16$. Node 4 is added to **Far**.

3. **Phase 3 Entry (Iteration 2):** Updating keys from **tent** yields:

$$\text{far_queue} = [3, 4]$$

$$\text{far_keys} = [6, 16] \quad (\text{Node 3 has updated key 6})$$

4. **Ghost Pruning Action:** Compare keys against $\text{last_cutoff} = 10$.

$$\text{Key of Node 3} = 6 \leq 10 \implies \text{Node } \mathbf{3} \text{ IS A GHOST (PRUNED!)}$$

5. **Cleaned Queue State:**

$$\text{far_queue} = [4]$$

$$\text{far_keys} = [16] \implies D_{\min} = 16, \Delta = 0$$

Impact If ghost entries are not pruned, $D_{\min}$ is incorrectly set to 6 ($\leq \text{last_cutoff}$). This produces negative or corrupted Δ values and re-extracts Node 3 into **Near**, leading to redundant kernel execution and non-deterministic trace logs.

Appendix: A Tutorial on Essential NVIDIA Thrust Primitives

NVIDIA Thrust is a C++ template library for CUDA that provides high-level parallel algorithms and data structures. It allows developers to execute parallel operations—such as sorting, searching, deduplication, and memory transfers—directly on GPU memory using an iterator-based interface.

This appendix provides a generic, standalone tutorial for the core Thrust primitives used in modern GPU memory management pipelines.

1. Pointer Encapsulation (`thrust::device_ptr`)

Raw device memory allocated using standard CUDA runtime APIs (`cudaMalloc`) cannot be passed directly into Thrust algorithms. Wrapping a raw pointer with `thrust::device_ptr` converts it into a Thrust-compatible device iterator without making a copy or allocating additional GPU memory.

```
#include <cuda_runtime.h>
#include <thrust/device_ptr.h>

// 1. Allocate raw device memory
int* d_raw = nullptr;
cudaMalloc(&d_raw, 5 * sizeof(int));

// 2. Wrap the raw pointer in a thrust::device_ptr
thrust::device_ptr<int> dev_ptr(d_raw);

// Now dev_ptr can be used as an iterator:
// dev_ptr, dev_ptr + 5
```

2. Key-Value Parallel Sorting (`thrust::sort_by_key`)

`thrust::sort_by_key` performs a parallel sort on a key range while synchronously permuting a corresponding value range. The elements in the value array are moved to match the new position of their associated keys.

```
#include <thrust/device_ptr.h>
#include <thrust/sort.h>

// Input arrays:
// Keys:   [ 30, 10, 20 ]
// Values: [100, 200, 300 ]

thrust::device_ptr<int> dev_keys(d_keys);
thrust::device_ptr<int> dev_vals(d_vals);

// Sort keys in ascending order while permuting values
thrust::sort_by_key(dev_keys, dev_keys + 3, dev_vals);

// Resulting arrays:
// Keys:   [ 10, 20, 30 ]
// Values: [200, 300, 100 ]
```

3. Adjacent Key Deduplication (`thrust::unique_by_key`)

`thrust::unique_by_key` reduces contiguous sequences of duplicate keys down to a single instance, simultaneously removing the matching values.

Note: To deduplicate a fully unsorted dataset, the array must first be sorted by key using `thrust::sort_by_key` so that all identical keys are adjacent.

```
#include <thrust/device_ptr.h>
#include <thrust/unique.h>

// Pre-sorted Input Arrays:
// Keys:   [ 1,  1,  2,  3,  3 ]
// Values: [10, 15, 20, 25, 30 ]

thrust::device_ptr<int> dev_keys(d_keys);
thrust::device_ptr<int> dev_vals(d_vals);

// Collapse duplicate adjacent keys and their corresponding values
auto new_end = thrust::unique_by_key(dev_keys, dev_keys + 5, dev_vals);

// Compute the new size of the array
int new_size = new_end.first - dev_keys; // new_size = 3

// Resulting arrays (first 3 elements):
// Keys:   [ 1,  2,  3 ]
// Values: [10, 20, 25 ]
```

4. Binary Search Boundary Search (`thrust::upper_bound`)

`thrust::upper_bound` performs a parallel binary search over a sorted range to find the last position where a target value can be inserted without violating the sorted order. It returns an iterator pointing to the first element that is strictly greater than the search value.

```
#include <thrust/device_ptr.h>
#include <thrust/binary_search.h>

// Sorted Array: [ 10, 20, 20, 30, 40 ]
// Target Value: 20

thrust::device_ptr<int> dev_keys(d_keys);

// Find iterator to first element strictly > 20
thrust::device_ptr<int> bound_ptr = thrust::upper_bound(dev_keys, dev_keys
+ 5, 20);

// Calculate index offset
int count = bound_ptr - dev_keys; // count = 3 (pointing to element 30)
```

5. Memory Transfers and Compaction (`thrust::copy`)

`thrust::copy` copies a range of elements from a source sequence to a destination sequence. It supports device-to-device, host-to-device, and device-to-host transfers, handling iterator offsets safely.

```
#include <thrust/device_ptr.h>
#include <thrust/copy.h>

// Shift elements left: move elements from index 2..4 to index 0..2
thrust::device_ptr<int> dev_data(d_data);

thrust::copy(dev_data + 2, dev_data + 5, dev_data);

// Elements previously at indices [2, 3, 4] are now at indices [0, 1, 2]
```

6. Primitive Summary Reference

Table 2: Generic Thrust Parallel Algorithms Summary

Function	Primary Operation	Time Complexity
<code>thrust::device_ptr</code>	Wraps raw CUDA allocation into an iterator	$O(1)$
<code>thrust::sort_by_key</code>	Parallel key-value sort using Radix Sort	$O(N \log N)$
<code>thrust::unique_by_key</code>	Removes adjacent duplicate keys and values	$O(N)$
<code>thrust::upper_bound</code>	Parallel binary search for strict upper threshold	$O(\log N)$
<code>thrust::copy</code>	Device-to-device array copying and shifting	$O(N)$